Overview

Computer can only understand one language, **machine language**. Machine language is very difficult to learn because it consists only of the binary numbers 0 and 1.

So we use languages that we know as well as there are some ways of converting this to machine language. It is very hard to convert natural languages, so special programming languages have been developed similar to natural languages (specially in English). Therefore these are easy to learn. These programming languages are used to develop computer programs that give the computer a detailed set of instructions to execute.

It is very tempting to start entering your program into the computer as soon as you have some idea of how to write it. Instead, you need to analyse the problem and its solution before you write any program instructions.

It has been clearly demonstrated that time spent defining the problem to be solved and then designing the solution leads to more efficient programming and fewer errors in the program.

Program Development Cycle

The task of creating a program that meets the user's requirements and is adaptable is a complex and difficult task unless it is organised. This is done using a cycle of well-defined steps to create simpler parts to reduce the complexity.

A number of stages can be identified in the program development process. These stages are referred to as the **Program Development Cycle** to emphasise the point that it may be necessary to return to an earlier stage if errors or ambiguities are detected.

Six stages of program development can be identified:

1. Problem definition.
2. Development of a solution.
3. Testing of the solution.
4. Coding of the solution.
5. Testing the program.
6. Maintenance and documenting the program.

1. Problem Definition

This stage is the most important because all subsequent stages are dependent on it. Problem definition involves analysing the problem until a clear understanding is reached. Without a clear understanding of the problem, time and effort spent on later stages can be wasted. It may be necessary to have a discussion with the person who proposed the problem to clarify issues.

In a business environment the programming problem is generally presented as a **program specification**, which sets out

i) Input,
ii) Processing
iii) Output requirements and
iv) Other requirements of the program.

The program specification should also include known results for given inputs to the program. Sometimes these known results are termed as test cases. These known results can be used as a guide to the successful solution of the problem.

An **IPO** chart is a tool that can be used to help document the problem definition.

> An IPO chart is a table that specifies the input, processing and output requirements of the program.

IPO charts are sometimes called **defining diagram**.

An accessory to the IPO chart is a list of names for each of the input and output data storage. These names are easier to use than a full text description of an item and can lead to the removal of ambiguity. This list is referred to as the **Initial Data Dictionary**. Other data items, which are neither input nor output, are added to this list as the processing steps are developed. The complete list of data items used in a program is referred to as the **Data Dictionary** for the program.

2. Development of a Solution

The technique of **top-down design** in the development of a solution involves breaking the problem down into a number of tasks to establish an outline of the solution, and then breaking these tasks down into sub-tasks until the solution of each task is manageable.

The solution outline can be represented by a **hierarchy or structure chart**. Hierarchy charts show the relationships between sections in a program.

The set of logical steps that describe the actions to be carried out in each task to solve the overall problem is called an **algorithm**.

There are many different techniques used to represent an algorithm. Algorithms can be **diagrammatic** or **descriptive**.

Diagrammatic algorithms include:
1. Flowcharts.
2. Nassi-Shneiderman diagrams.
3. Action diagrams.

Descriptive algorithms include:
1. Pseudocode.
2. Point form.
3. Structured English.

6. Maintenance of the Program and Documentation

Documentation consists of the written materials produced at each stage of the program development process. It is a cumulative process that starts with the initial statement of the problem, adds the details of the solution design, then the coding and details of the testing.

One document that is developed over several of the stages in the program development cycle is the **Data Dictionary**. This is an alphabetic list of the names of all the data items used by the program. It includes a description of the data item and the type of data. The initial data dictionary from the problem definition stage is added to during algorithm development and during coding as extra data items are used in the steps to convert the input to output.

The data dictionary is provided both as written documentation and as a series of comments in the first part of the program. The written document is used as a reference for data items in the algorithm; the data dictionary in the program provides the programmer, and other programmers, with a reference section to ensure consistency in the use of data within the program.

Once the program has been thoroughly tested, the documentation should be reviewed to ensure consistency between the materials produced at each stage.

Documentation for a program consists of:

1. Program specification.
2. IPO chart.
3. Hierarchy or structure chart.
4. Algorithm in pseudocode or NS diagrams or flowcharts.
5. Test data and desk check.
6. Program listing.
7. Results of testing the program.

It is likely that the program will be changed at some time after it has been successfully designed and tested. Errors may occur or modifications may be requested. As changes are made they should be documented and the materials from each stage modified to reflect those changes.

Problem Definition

A program specification is a formal way of documenting what a computer program is to do. The format of a program specification should contain the following:

1. A general **DESCRIPTION** of the program i.e. what it is required to do.
2. Program **INPUTS** - a list of data items which are to be provided as inputs to the program and where these inputs are coming from e.g. keyboard, file etc.
3. The **PROCESSING** requirements - a description of how the program inputs are to be processed so that the required program outputs are produced.
4. Program **OUTPUTS** - a list of those results which are to be produced by the program and where the outputs are going e.g. screen, printer, file etc.
5. The **OVERALL TASKS**, i.e. the broad solution steps (the highest level of top-down design) or an overall view of the tasks the program will have to perform.

The purposes of the program specification are:

1. It helps to clarify what the program is required to do.
2. Any problem ambiguities are removed.
3. It gives the programmer an excellent starting point for thinking about the details of the algorithm.

Below is an example of a program specification.

Example 1.1

DESCRIPTION

The Computer Disk Supply Company has made a request for a program to handle the ordering of their floppy disks. The diskettes are supplied in boxes of 10 and prices vary according to size and density of the diskettes. The program is to ask for the details of an order then give the cost of the order entered. After an order is entered the user is to be given the chance to continue to enter another order (by pressing the enter key) or to terminate the input of orders by entering the word END. Once the user enters END, some statistics on all orders entered are to be provided.

VARIABLE	DESCRIPTION	SOURCE
CLIENT	The name of the person/company ordering	From keyboard

PROCESSING

1. Validate each of the inputs as they are entered. If an invalid value is entered an error message should be printed and a new value requested until a valid value is entered.
2. Calculate the order cost as follows:
   - 3.5" diskettes cost $20 per box for double density.
   - 5.25" diskettes cost $10 per box for double density.
   - 8" diskettes cost $15 per box for double density.
   - For 3.5" and 5.25" diskettes add $3 per box for high density.
3. Statistics includes:
   - The number of orders processed.
   - The number of boxes of each size of diskette sold (all densities combined).
   - The cost of all orders.

Note: The precise details of converting a temperature in degrees Celsius to a temperature in degrees Fahrenheit are not required at this stage. It is sufficient to describe **what** is to be done, not **how** it will be done.

The IPO chart should now be analysed and modified until the following questions are satisfied:

1. Are all the inputs required for the processing steps specified?
2. Are all of the required outputs specified?
3. Do the processing steps transform input to output?

Exercise 2

Problem 1:

Using an IPO Chart, determine how an employee's gross pay and net pay is determined. The employees would need to be prompted to enter their hours worked and the hourly rate. The result would be displayed on the screen.

where
gross pay = hours worked x hourly rate
tax amount = 10% of gross pay
net pay = gross - tax amount

Problem 2:

A program is required to perform arithmetic calculations on two integers. The program should prompt for the input of the two integer numbers and display the following on the screen:

a. The sum of the two numbers.
b. The difference of the two numbers.
c. The product of the two numbers.
d. The quotient of the two numbers.

Develop an IPO Chart.